1-1 Introduction

Procedural Language: It is a type of programming language that follows a procedure; set of commands or guidelines that must be followed for smooth execution of the program. It works on step by step basis. It requires the user to tell not only **What to do** but also **How to do it**. Its basic idea is to have a program specify the sequence of steps that implements a particular algorithm. Hence, Procedural languages are based on algorithms.

Object-Oriented Programming or **OOP** refers to languages that use objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming.

The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

1-2 C++ Classes and Objects

In Object Oriented Programming, classes and objects are basic concepts of that are used to represent real-world concepts and entities.

- A class is a template to create objects having similar properties and behavior, or in other words, we can say that a class is a blueprint for objects.
  - An object is an instance of a class. For example,

  Class :
  - The class represents the blueprint of a house. It defines the structure and properties such as the number of rooms, color, and number of floors.

  Object:
  - The object represents the real house built from that blueprint. Each house created from the blueprint is an object..

C++ Classes

A class is a **user-defined data type,** which holds its own data members and member functions that can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object.

Create a Class: A class must be defined before its use. C++ class is defined using the keyword **class** keyword.

Generally a class specification has two parts:-

a) Class declaration
b) Class function definition.

the class declaration describes the type and scope of its members. The class function definition describes how the class functions are implemented.

Syntax:-

```
class class-name
{
private:
    variable declarations;
    function declaration ;
public:
    variable declarations;
    function declaration;
};
```

- The members that have been declared as private can be accessed only from with in the class.
- public members can be accessed from outside the class also.

- The data hiding is the key feature of oops.

- The use of keywords private is optional by default, the members of a class are private. The variables declared inside the class are known as data members and the functions are known as members mid the functions. Only the member functions can have access to the private data members and private functions. However, the public members can be accessed from the outside the class. The binding of data and functions together into a single class type variable is referred to as encapsulation.

- Syntax:-

```
class item
{
int member;
float cost;
public:
void getldata (int a ,float b);
void putdata (void);
}
```

The class item contains two data members and two function members, the data members are private by default while both the functions are public by declaration. The function getdata() can be used to assign values to the member variables member and cost, and putdata() for displaying their values . These functions provide the only access to the data members from outside the class.

Example: This example illustrates a class definition or template.

Our first program contains a class and two objects of that class. Although it's simple, the program demonstrates the syntax and general features of classes in C++. Here's the listing for the SMALLJOB program:

```
#include <iostream>
using namespace std;
class smalljob
{
private:
        int somedata;
public:
                void setdata(int d)
                { somedata =d;
                }
                void showdata()
                { cout <<"Data is "<< somedata<<endl;
                }
};


int main()
{
        smalljob s1, s2;      // define two objects of class smalljob
        s1.setdata(1066);     // call member functions to set data
        s2.setdata(1776);

        s1.showdata();      // call member function to display data
```

```
        s2.showdata();
        return 0;

}
```

- The class **smalljob** defined in this program contains one data item and two member functions.
- The two member functions provide the only access to the data item from outside the class. The first member function sets the data item to a value, and the second displays the value.
- Placing data and functions together into a single entity is a central idea in object oriented programming. This is shown in Figure 1.

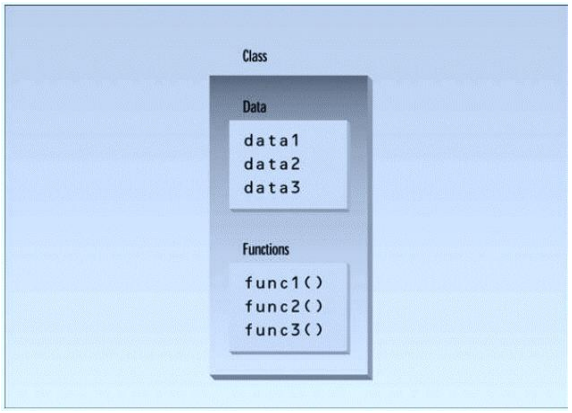

FIGURE 1: Classes contain data and functions.

1-2 Classes and Objects

An object has the same relationship to a class that a variable has to a data type. An object is said to be an instance of a class. In SMALLJOB, the class—whose name is smalljob—is defined in the first part of the program. Later, in main(), we define two objects—s1 and s2—that are instances of that class. Each of the two objects is given a value, and each displays its value. Here's the output of the program:

```
Data is 1066 ⟶ object S1 displayed this data value

Data is 1776 ⟶ object S2 displayed this data value
```

1-3 Private and Public:

The body of the class contains two unfamiliar keywords: private and public. What is their purpose? A key feature of object-oriented programming is *data hiding*. This term means that data is concealed within a class so that it cannot be accessed mistakenly by functions outside the class. The primary mechanism for hiding data is to put it in a class and make it private. Private data or functions can only be accessed from within the class. Public data or functions, on the other hand, are accessible from outside the class. This is shown in Figure 2.

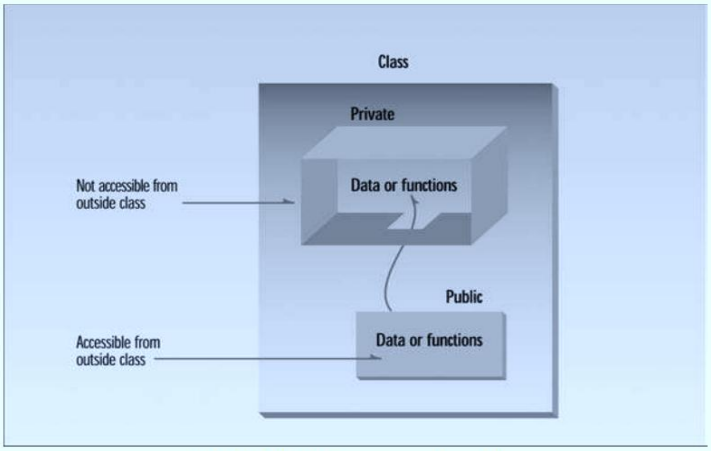

FIGURE 2 *Private and public.*

1-4 Using the Class

Now that the class is defined, let's see how main() makes use of it. We'll see how objects are defined, and, once defined, how their member functions are accessed.

1-4-1 Defining Objects

The first statement in main() smalljob s1, s2; defines two objects, s1 and s2, of class smalljob. Remember that the definition of the class smalljob does not create any objects. It only describes how they will look when they are created, just as a structure definition describes how a structure will look but doesn't create any structure variables. It is objects that participate in program operations. Defining an object is similar to defining a variable of any data type: Space is set aside for it in memory. Defining objects in this way means creating them. This is also called instantiating them. The term instantiating arises because an instance of the class is created. An object is an instance of a class. Objects are sometimes called instance variables.

1-4-2 Calling Member Functions :

The next two statements in main() call the member function setdata():

```
s1.setdata(1066);

s2.setdata(1776);
```

Similarly, the following two calls to the showdata() function will cause the two objects to display their values:

```
s1.showdata();

s2.showdata();
```

To use a member function, the dot operator (the period) connects the object name and the member function. The syntax is similar to the way we refer to structure members, but the parentheses signal that we're executing a member function rather than referring to a data item. (The dot operator is also called the class member access operator.)